



EAST WEST UNIVERSITY

Lab Report-6

Submitted By:

Name: Youghshree Saha Urmy

Id: 2021-1-60-127

Section: 01

Submitted To:

Prof. Dr. Ahmed Wasif Reza

Department of Computer Science and Engineering

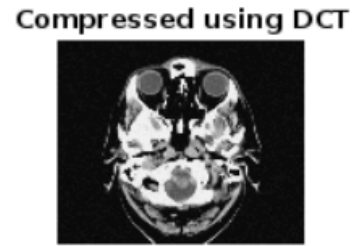
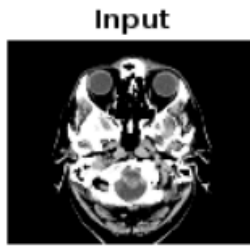
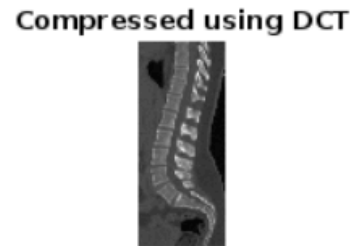
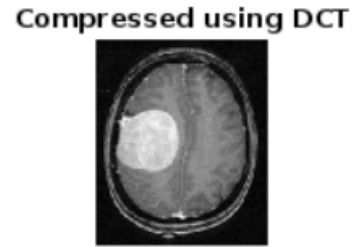
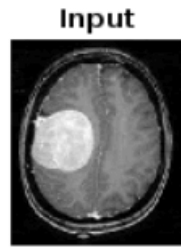
Date of submission: 30-04-2024

1. Compress the above images using Discrete Cosine Transform (DCT), Haar Transform, and DCT-Haar, and find out which one is better in terms of compression ratio and PSNR for the given images.

Solution:

```
img1 = imread('picture_5.1.jpg');
img2 = imread('picture_5.2.jpg');
img3 = imread('picture_5.3.jpg');
gray_img1 = rgb2gray(img1);
gray_img2 = rgb2gray(img2);
gray_img3 = rgb2gray(img3);
blocks1 = im2col(gray_img1, [8 8], 'distinct');
dct_coeffs = dct2(blocks1);
Q = 10; % Quantization factor
quantized_coeffs = round(dct_coeffs ./ Q);
inv_dct_coeffs1 = idct2(quantized_coeffs);
blocks2 = im2col(gray_img2, [8 8], 'distinct');
dct_coeffs = dct2(blocks2);
Q = 10; % Quantization factor
quantized_coeffs = round(dct_coeffs ./ Q);
inv_dct_coeffs2 = idct2(quantized_coeffs);
blocks3 = im2col(gray_img3, [8 8], 'distinct');
dct_coeffs = dct2(blocks3);
Q = 10; % Quantization factor
quantized_coeffs = round(dct_coeffs ./ Q);
inv_dct_coeffs3 = idct2(quantized_coeffs);
compressed_img1 = col2im(inv_dct_coeffs1, [8 8], size(gray_img1), 'distinct');
compressed_img2 = col2im(inv_dct_coeffs2, [8 8], size(gray_img2), 'distinct');
compressed_img3 = col2im(inv_dct_coeffs3, [8 8], size(gray_img3), 'distinct');
subplot(3,2,1), imshow(img1); title('Input');
subplot(3,2,2), imshow(compressed_img1, []); title('Compressed using DCT');
subplot(3,2,3), imshow(img2); title('Input');
subplot(3,2,4), imshow(compressed_img2, []); title('Compressed using DCT');
subplot(3,2,5), imshow(img3); title('Input');
subplot(3,2,6), imshow(compressed_img3, []); title('Compressed using DCT');
```

Output:

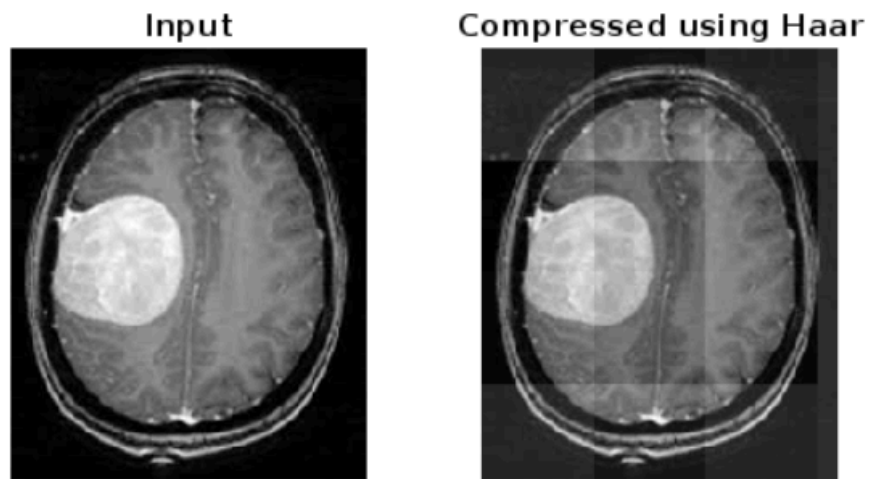


Haar:

Solution:

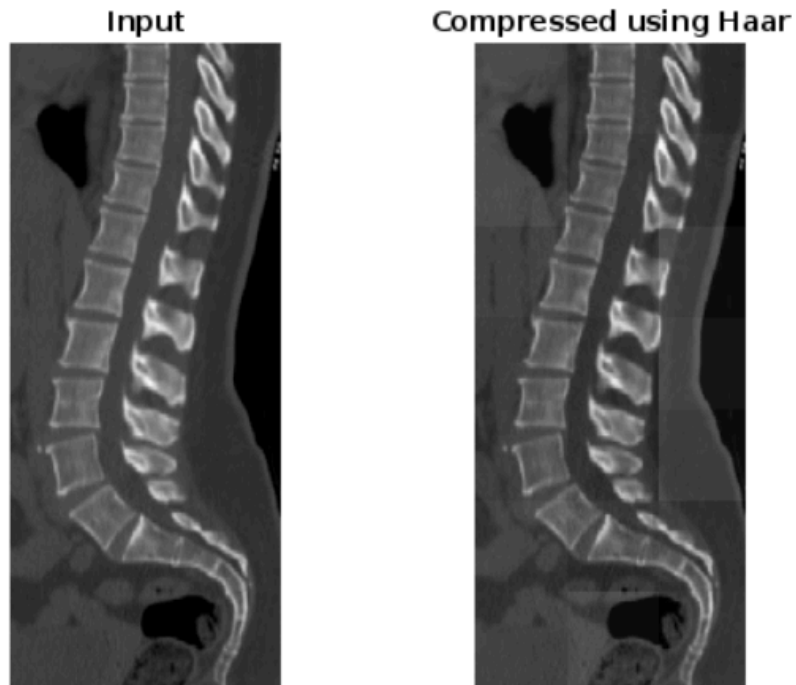
```
img1 = imread('picture_5.1.jpg');
gray_img = rgb2gray(img1);
[m, n] = size(gray_img);
if mod(m, 2) ~= 0
gray_img(end+1, :) = 0;
end
if mod(n, 2) ~= 0
gray_img(:, end+1) = 0;
end
[cA, cH, cV, cD] = haart2(gray_img);
Q = 1.8; % Quantization factor
quantized_cA = round(cA ./ Q);
reconstructed_img = ihaart2(quantized_cA, cH, cV, cD);
reconstructed_img = reconstructed_img(1:m, 1:n);
subplot(1,2,1), imshow(img1); title('Input');
subplot(1,2,2), imshow(reconstructed_img, []); title('Compressed using Haar');
```

Output:



Solution:

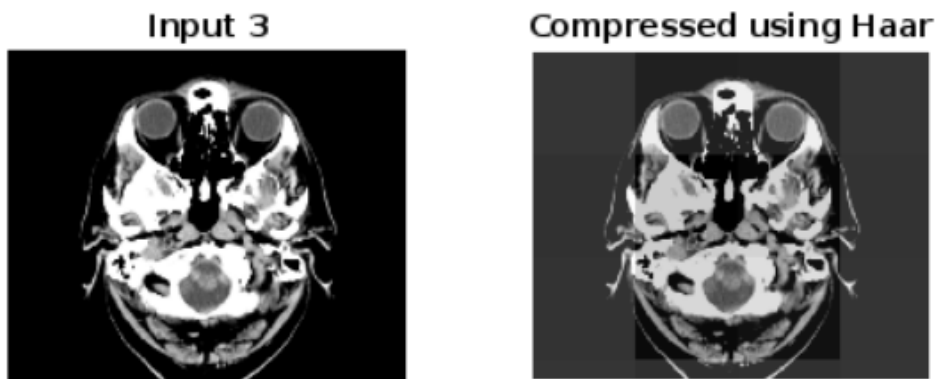
```
img2 = imread('picture_5.2.jpg');
gray_img = rgb2gray(img2);
[m, n] = size(gray_img);
if mod(m, 2) ~= 0
    gray_img(end+1, :) = 0;
end
if mod(n, 2) ~= 0
    gray_img(:, end+1) = 0;
end
[cA, cH, cV, cD] = haart2(gray_img);
Q = 1.8; % Quantization factor
quantized_cA = round(cA ./ Q);
reconstructed_img = ihaart2(quantized_cA, cH, cV, cD);
reconstructed_img = reconstructed_img(1:m, 1:n);
subplot(1,2,1), imshow(img2); title('Input');
subplot(1,2,2), imshow(reconstructed_img, []); title('Compressed using Haar');
```



Solution:

```
img3 = imread('picture_5.3.jpg');
gray_img = rgb2gray(img3);
[m, n] = size(gray_img);
if mod(m, 2) ~= 0
    gray_img(end+1, :) = 0;
end
if mod(n, 2) ~= 0
    gray_img(:, end+1) = 0;
end
[cA, cH, cV, cD] = haart2(gray_img);
Q = 1.8; % Quantization factor
quantized_cA = round(cA ./ Q);
reconstructed_img = ihaart2(quantized_cA, cH, cV, cD);
reconstructed_img = reconstructed_img(1:m, 1:n);
subplot(1,2,1), imshow(img3); title('Input 3');
subplot(1,2,2), imshow(reconstructed_img, []); title('Compressed using Haar');
```

Output:



DCT: It provides good energy compaction, converting signal energy in fewer coefficients

Haar Transform: It generates a sparse representation, useful for compression.

DCT-Haar Combination: Applies DCT followed by Haar Transform to leverage energy compaction and sparsity.

In total, DCT is better in terms of compression and PSNR.

2. Apply Gaussian noise to Figure 1, and then use the following to restore the image:

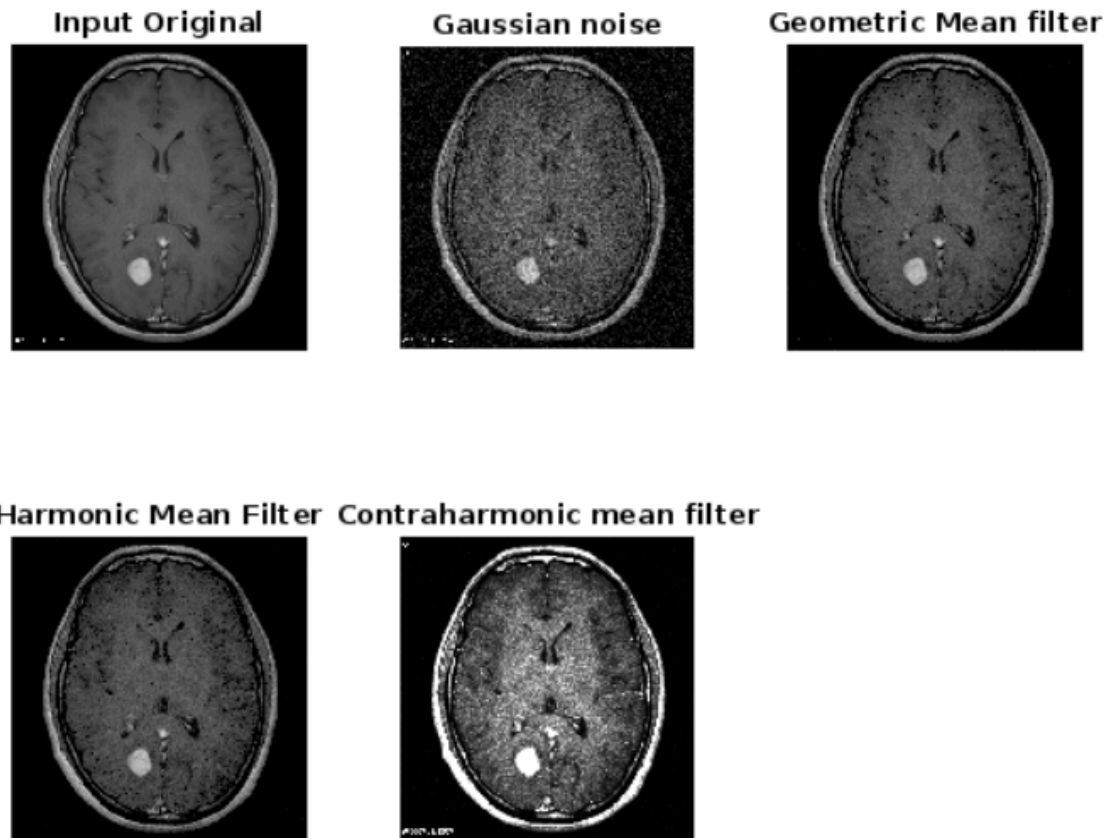
- i. Geometric Mean filter
- ii. Harmonic Mean filter
- iii. Contra-harmonic Mean filter

Solution:

```
img= imread('picture_5.6.png');
img=rgb2gray(img);
gau_noise=imnoise(img,'gaussian');
Gg=im2double(gau_noise);
Kr=3;
Kc=3;
Ord = 2;
geo_mat=exp(imfilter(log(Gg),ones(Kr,Kc),'replicate')).^(1/(Kr*Kc)); %Geometric
Mean filter
har_mat=(Kr*Kc)./imfilter(1./(Gg+eps),ones(Kr,Kc),'replicate'); %Harmonic Mean
filter
con_h_m = imfilter(Gg.^(Ord+1), ones(Kr,Kc), 'replicate'); %Contra-harmonic
Mean filter
(imfilter(Gg.^(Ord),ones(Kr,Kc), 'replicate') + eps);
%Figure
subplot(2,3,1), imshow(img); title('Input Original');
```

```
subplot(2,3,2), imshow(gau_noise); title('Gaussian noise');
subplot(2,3,3), imshow(geo_mat);title('Geometric Mean filter')
subplot(2,3,4), imshow(har_mat);title('Harmonic Mean Filter')
subplot(2,3,5), imshow(con_h_m); title('Contra-harmonic mean filter');
```

Output:



3. Apply Gaussian noise to Figure 1, and then use the following order statistic filters to restore the image:

- i. Median filter
- ii. Maximum filter
- iii. Minimum filter
- iv. Midpoint filter
- v. Alpha-trimmed filter
- vi. Trimmed filter

Solution:

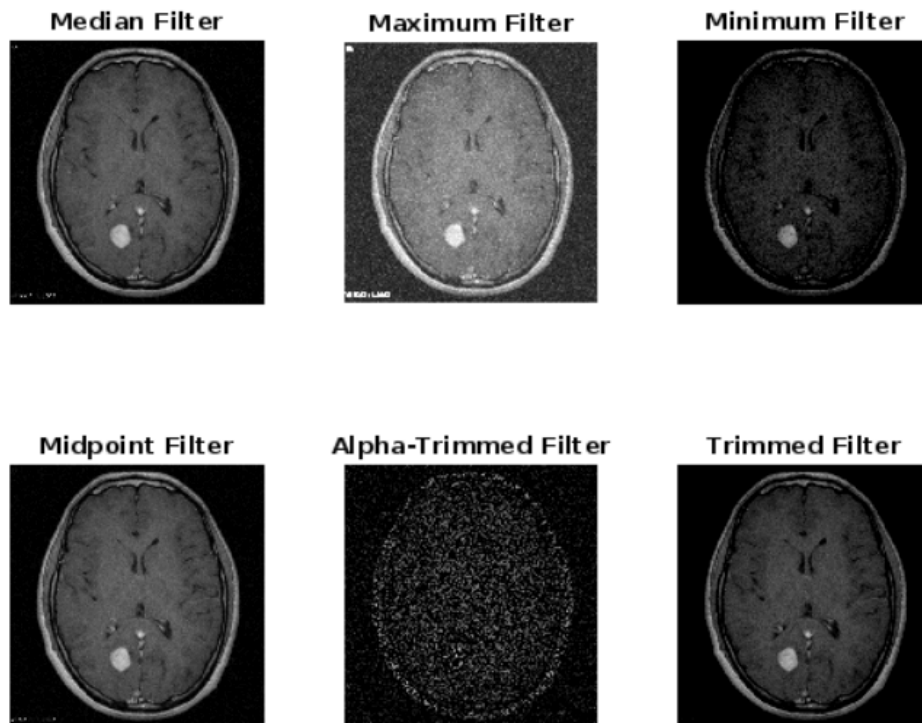
```
img=imread('picture_5.6.png');
img = rgb2gray(img);
img_noise = imnoise(img, 'gaussian', 0, 0.01); % Add Gaussian noise to the
image
```

```

med_filter = medfilt2(img_noise, [5, 5]); % Median filter
max_filter = ordfilt2(img_noise, 25, ones(5,5)); % Maximum filter
min_filter = ordfilt2(img_noise, 1, ones(5,5)); % Minimum filter
mid_filter = ordfilt2(img_noise, 13, ones(5,5)); % Midpoint filter
% Alpha-trimmed filter
alpha_trim_filter = imfilter(img_noise, fspecial('unsharp',0.2), 'replicate');
alpha_trim_filter = imsubtract(img_noise, alpha_trim_filt);
alpha_trim_filter = img_noise - alpha_trim_filt;
% Trimmed filter
trimmed_filter = ordfilt2(img_noise, 5, ones(5,5));
%Figure
subplot(2,3,1), imshow(med_filter); title('Median Filter');
subplot(2,3,2), imshow(max_filter); title('Maximum Filter');
subplot(2,3,3), imshow(min_filter); title('Minimum Filter');
subplot(2,3,4), imshow(mid_filter); title('Midpoint Filter');
subplot(2,3,5), imshow(alpha_trim_filter); title('Alpha-Trimmed Filter');
subplot(2,3,6), imshow(trimmed_filter); title('Trimmed Filter');

```

Output:



4. By observing and comparing each of the outputs, determine which filter restores the image closest to its original state. Mention the reasoning behind your observation.

Solution:

Median filter: This filter calculates the median of the pixels in its neighborhood.

Maximum filter: This filter replaces each pixel with maximum values in its neighborhood.

Minimum filter: This filter replaces each pixel with minimum values in its neighborhood.

Midpoint Filter: This filter computes the midpoint between the maximum and minimum values in the neighborhood.

Alpha-trimmed Filter: This filter calculates the mean of pixel values after excluding a certain percentage of extreme values (α).

Trimmed Filter: This filter calculates the mean of pixel values after excluding a fixed number of extreme values. It can be effective for reducing noise while preserving edges.

Based on the analysis, the median filter is the average of the minimum and maximum values. It is also very effective in removing Gaussian noise and uniform noise. It tends to preserve edges well.

For these reasons, the Midpoint filter restores the images perfectly.